

UNIVERSIDAD
DE SANTIAGO
DE CHILE

UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA MECÁNICA

INFORME LABORATORIO N°1

Manipulador Robótico

Asignatura: Fundamentos de Robótica

Profesores: Diego Luna; Arturo Álvarez

Integrantes: Martin Barrezueta
Ignacio Espinoza
Emilio Rojas

Fecha: 06-11-2026

ÍNDICE DE CONTENIDOS

Resumen 1
1. Introducción 1
    1.1 Objetivos 1
2. Marco teórico 1
3. Presentación del problema 2
4. Solución del problema 3
    4.1 Diseño y equipos 3
    4.2 Metodología 4
    4.3 Resultados 5
5. Conclusiones 5
6. Referencias 6
Anexo I. Rutina de código I-1
    I.1 Descripción I-1
    I.2 Parámetros I-1
    I.3 Código I-2
Anexo II. Desarrollo de algoritmo D-H para manipulador robótico II-1
    II.1 Parámetros y convención II-1
    II.2 Tabla D-H y matriz de transformación II-1
    II.3 Algoritmo II-2
    II.4 Resultados II-2

ÍNDICE DE TABLAS

Tabla 1. Características técnicas de los equipos 3
Tabla 2. Ángulos articulares y de servo en los puntos de la rutina 5
Tabla 3. Parámetros de la rutina I-1
Tabla 4. Medidas del manipulador II-1
Tabla 5. Parámetros D-H del manipulador II-1
Tabla 6. Posiciones de las articulaciones en la pose inicial II-2

ÍNDICE DE FIGURAS

Figura 1. Esquema eléctrico del manipulador 4
Imagen 1. Brazo manipulador 2

RESUMEN

Se construyó un manipulador robótico impreso en 3D, con base, hombro, codo y pinza, accionado por servomotores y controlado por una ESP32 (*Ver imagen*). El brazo detecta cubos de 20 mm en tres posiciones conocidas con un sensor ultrasónico HC-SR04, los traslada y los apila en un punto final. La posición de la pinza se modeló con el método de Denavit-Hartenberg (D-H), y la cinemática inversa se resolvió de forma analítica para obtener los ángulos de cada servo. Una aplicación Android, conectada por WiFi, permite el control manual, la ejecución de la rutina y la visualización de las posiciones articulares. Los puntos de recogida y de depósito resultaron alcanzables dentro de los límites de los servos.

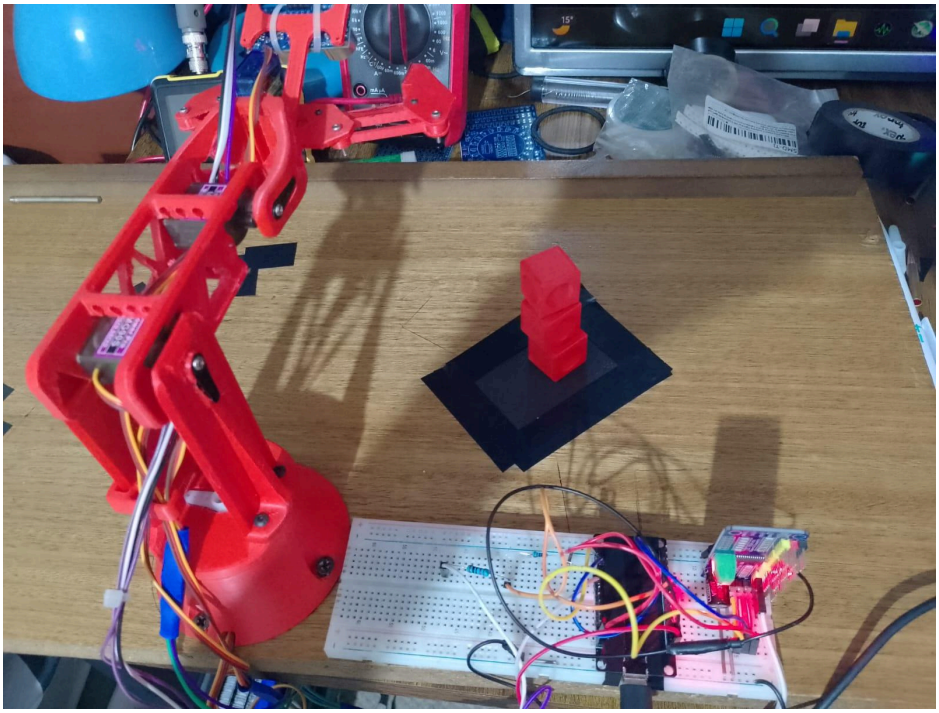


Imagen 1. Brazo manipulador

1. INTRODUCCIÓN

Los manipuladores robóticos combinan sistemas electromecánicos, automatización y software para ejecutar tareas con precisión y repetibilidad. Se emplean en las industrias automotriz, alimentaria, aeroespacial y médica, y una de sus aplicaciones habituales es el traslado y la organización de objetos.

Este laboratorio desarrolla una versión a pequeña escala: un brazo impreso en 3D que detecta cubos en posiciones establecidas y los apila en un punto determinado. Un modelo matemático calcula la posición de la pinza y permite controlarla.

1.1 Objetivos

Objetivo general. Implementar un robot manipulador a escala, impreso en 3D, capaz de trasladar y ordenar objetos mediante servomecanismos controlados por una ESP32.

Objetivos específicos.

1. Construir el manipulador con piezas impresas en 3D, unidas con pernos y tornillos.
2. Integrar los servomotores, la ESP32, el sensor ultrasónico y la comunicación WiFi.
3. Controlar el brazo de forma manual y automática desde una aplicación Android.
4. Modelar el manipulador con el algoritmo D-H y usar su cinemática inversa para posicionar la pinza.
5. Ejecutar rutinas automáticas de traslado y apilamiento de cubos de dimensiones conocidas.

2. MARCO TEÓRICO

Manipuladores robóticos. Son cadenas cinemáticas de eslabones rígidos unidos por articulaciones. Cada articulación aporta un grado de libertad (GDL) y el conjunto determina la posición y la orientación del elemento terminal [1], [2]. En este trabajo, la base, el hombro y el codo posicionan la pinza, y un cuarto servomotor la abre y la cierra.

Método D-H y cinemática inversa. La cinemática directa calcula la posición del elemento terminal a partir de los ángulos articulares, y la inversa obtiene los ángulos necesarios para alcanzar una posición dada [1], [2]. La convención D-H describe cada eslabón con cuatro parámetros: el ángulo θ, la distancia d sobre el eje z, la longitud a sobre el eje x y la torsión α. Su transformación homogénea es:

$$T(\theta, d, a, \alpha) = \begin{bmatrix} \cos\theta & -\sin\theta\,\cos\alpha & \sin\theta\,\sin\alpha & a\,\cos\theta \\ \sin\theta & \cos\theta\,\cos\alpha & -\cos\theta\,\sin\alpha & a\,\sin\theta \\ 0 & \sin\alpha & \cos\alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

Servomotores. Un servomotor posiciona su eje con una señal PWM de 20 ms de periodo (50 Hz); los pulsos de 0,5 a 2,5 ms recorren el rango completo. El torque es de 1,8 kg·cm en el SG90 y de 2,2 kg·cm en el MG90S [3].

Driver PCA9685. Controlador de 16 canales PWM de 12 bits (4.096 pasos) con interfaz I2C [4]. A 50 Hz, cada paso dura cerca de 4,9 µs, y dos hilos bastan para gobernar varios servos.

ESP32. Microcontrolador de doble núcleo con WiFi y Bluetooth, cuyos pines operan a 3,3 V y no toleran 5 V [5]. Con WiFi, la placa puede actuar como servidor TCP y recibir órdenes de texto desde un teléfono [6].

Sensor HC-SR04. Mide la distancia por tiempo de vuelo: un pulso de 10 µs en TRIG emite una ráfaga ultrasónica y el pulso en ECHO dura lo que tarda el eco en volver. La distancia es $d = t \cdot v / 2$, con $v \approx 343$ m/s a 20 °C [7]. Funciona a 5 V, por lo que ECHO requiere un divisor de tensión hacia la ESP32.

Alimentación. Los servomotores generan picos de corriente al arrancar o al hacer esfuerzo, con caídas de tensión y ruido. Por ello se alimentan aparte con 5 V regulados y un condensador en la línea, y comparten solo la tierra con la ESP32, que se alimenta por USB.

Aplicación móvil. Una aplicación Android escrita en Kotlin abre un socket TCP hacia la ESP32 e intercambia líneas de texto con órdenes y telemetría [8].

3. PRESENTACIÓN DEL PROBLEMA

Se requiere un brazo robótico que, con el modelo D-H, ubique su pinza en distintas posiciones para trasladar objetos desde puntos iniciales y apilarlos en un punto final. Los requerimientos son:

- trasladar objetos de dimensiones y peso conocidos desde puntos específicos del área de trabajo;
- detectar con el sensor ultrasónico si hay un bloque en cada punto;
- ejecutar una rutina que lleve cada bloque detectado desde su punto inicial Q_i hasta el punto final Q_f y lo apile;
- disponer de una aplicación móvil para interactuar con el manipulador;
- visualizar las posiciones de las articulaciones obtenidas con el algoritmo D-H.

Condiciones. Cubos de PLA de 20 mm de arista. Posiciones de recogida (centro del cubo respecto de la base, en mm): P_1 (165,5; 0), P_2 (150; 70) y P_3 (106,4; 126,8). Punto de depósito Q_f: (72,7; −155,9). Las tres posiciones de recogida están a 165,5 mm del eje de la base y el punto de depósito a 172,0 mm. El alcance máximo teórico del punto de agarre desde el eje del hombro es de 210,9 mm.

4. SOLUCIÓN DEL PROBLEMA

4.1 Diseño y equipos

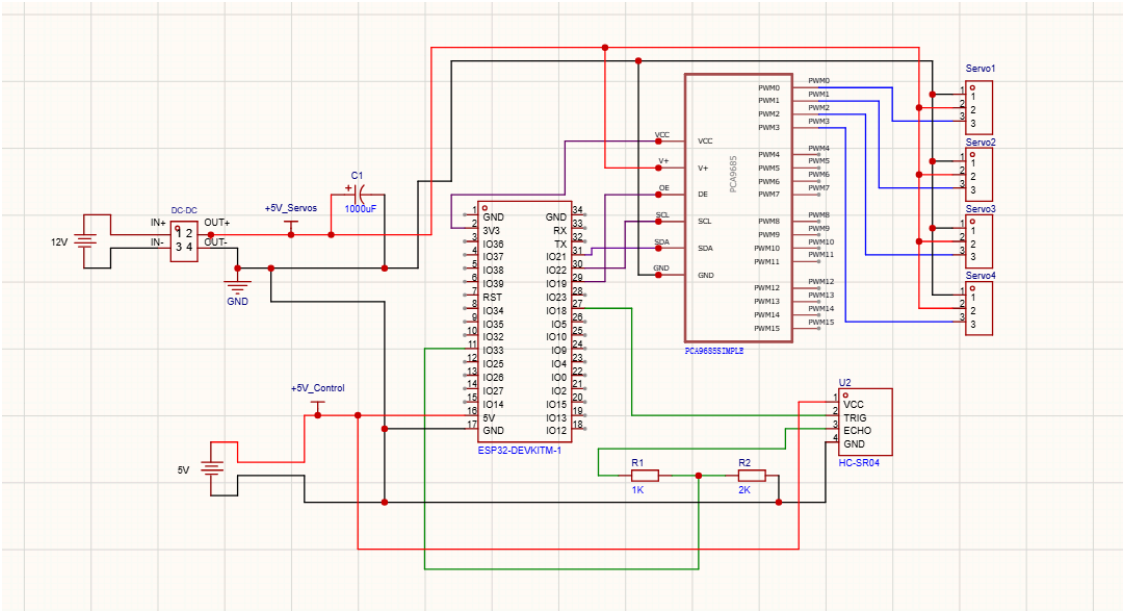
El manipulador se imprimió en PLA y sus eslabones se unieron con pernos y tornillos. El eje del hombro está a 128,8 mm de la mesa, el brazo mide 71,8 mm y el antebrazo 51,5 mm. El punto de agarre queda 87 mm por delante de la muñeca y 13 mm bajo la línea del antebrazo. La Tabla 1 resume los equipos y la Figura 1 su conexión.

Tabla 1. Características técnicas de los equipos

Equipo	Características	Conexión
Controlador ESP32-D	Doble núcleo, WiFi, lógica de 3,3 V; alimentado por USB	I2C en IO21 (SDA) e IO22 (SCL)
Driver PCA9685	16 canales PWM, 12 bits, I2C, 50 Hz; se usan los canales 0 a 3	VCC a 3V3 de la ESP32; V+ a la fuente de potencia
Servomotores SG90 (base y pinza)	Torque de 1,8 kg·cm	Señal a un canal del PCA9685; +5 V de la fuente de potencia
Servomotores MG90S (hombro y codo)	Torque de 2,2 kg·cm	Señal a un canal del PCA9685; +5 V de la fuente de potencia
Sensor HC-SR04	Ultrasónico, alimentación de 5 V	TRIG a IO18; ECHO a IO33 mediante divisor de 1 kΩ y 2 kΩ
Fuente de potencia	Cargador de celular de 5 V, con condensador en la línea	Alimenta solo los servomotores

La alimentación se divide en potencia (cargador de 5 V para los servomotores) y control (USB del PC para la ESP32 y, desde su pin VIN, para el HC-SR04). Solo se unen los GND: las líneas de +5 V de ambas fuentes nunca se conectan entre sí.

Figura 1. Esquema eléctrico del manipulador



4.2 Metodología

1. **Modelado.** Se definieron los parámetros D-H (Anexo II) y se programaron la cinemática directa y la inversa analítica.
2. **Calibración.** Cada servo se movió por separado para fijar su desfase y su sentido de giro respecto del ángulo articular, con la relación servo = desfase + signo·q (desfases de 90°, 70° y 77°; signos +1, −1 y +1). Los límites de servo adoptados son 0° a 180° en la base, 10° a 135° en el hombro y 1° a 170° en el codo.

3. **Detección.** Se midió la distancia sin cubo en cada posición y se fijó el umbral en 7,86 cm a partir de esos valores. Cada medición es la mediana de cinco lecturas, y hay objeto si la distancia no supera el umbral.
4. **Validación previa.** Antes de mover el brazo se comprueba que la cinemática inversa tenga solución, dentro de los límites de los servos, en cada punto y en cada tramo entre puntos. Si alguno falla, la rutina no parte.
5. **Trayectorias.** La pinza avanza por puntos intermedios cada 2 mm, calculados con la cinemática inversa: en arco a radio constante entre posiciones y en vertical al subir o bajar. El arranque y el frenado son suaves, con pausas de 15 ms por paso sin carga y de 25 ms con carga. Si un servo cambia más de 10° entre dos pasos, el movimiento se detiene.
6. **Rutina.** La pinza recorre las posiciones 1 a 3 y mide en cada una. Si detecta un cubo, lo toma, lo lleva a la pila y lo suelta 5 mm sobre el nivel correspondiente. Cada nivel sube 20 mm. Al terminar, el brazo vuelve a su posición de inicio (Anexo I).
7. **Aplicación.** La ESP32 se conecta a la red WiFi, o crea una propia si no la encuentra, y abre un servidor TCP en el puerto 8080. La aplicación envía las órdenes HOME, AUTO, STOP y S,canal,ángulo, y recibe el estado, los ángulos de los servos y las posiciones D-H de las articulaciones. STOP detiene el movimiento en cualquier instante.

4.3 Resultados

La Tabla 2 entrega los ángulos que resuelve la cinemática inversa para los puntos de recogida y de depósito. Los límites de servo se cumplen en todos ellos, y también en los puntos de aproximación, de escaneo y sobre la pila.

Tabla 2. Ángulos articulares y de servo en los puntos de la rutina
(coordenadas en mm; ángulos en grados; cubos de 20 mm)

Punto	(x, y, z)	q_1, q_2, q_3	Servos: base, hombro, codo
Recogida 1	(165,5; 0; 10)	0,00; −14,65; −26,33	90,0; 84,6; 50,7
Recogida 2	(150; 70; 10)	25,02; −14,68; −26,28	115,0; 84,7; 50,7
Recogida 3	(106,4; 126,8; 10)	50,00; −14,68; −26,28	140,0; 84,7; 50,7
Depósito, nivel 1	(72,7; −155,9; 15)	−65,00; −16,62; −20,11	25,0; 86,6; 56,9
Depósito, nivel 2	(72,7; −155,9; 35)	−65,00; 2,10; −40,62	25,0; 67,9; 36,4
Depósito, nivel 3	(72,7; −155,9; 55)	−65,00; 15,94; −52,83	25,0; 54,1; 24,2

El desarrollo del algoritmo y la verificación de la cinemática directa se presentan en el Anexo II.

5. CONCLUSIONES

El modelo D-H permitió calcular los ángulos de los servos para cada punto de recogida y de depósito, y los seis puntos de la Tabla 2 resultaron alcanzables dentro de los límites

configurados. Cumple así el planteamiento del problema en lo que respecta al posicionamiento de la pinza.

Comparación entre resultados. Los tres puntos de recogida comparten el radio de 165,5 mm, por lo que exigen casi los mismos ángulos de hombro y codo (−14,7° y −26,3°) y solo cambia la base (0°, 25,0° y 50,0°). En la pila, el servo del hombro disminuye 18,7° y 13,8° y el del codo 20,5° y 12,2° al subir cada nivel de 20 mm. El punto más exigente en alcance es la posición de escaneo 2, a 209,3 mm del eje del hombro, que fija hasta dónde pueden ubicarse las posiciones de recogida.

Aporte personal. En una prueba de esfuerzo, un servo SG90 no tuvo torque suficiente y otro se dañó. El torque de los servos y el peso de los eslabones limitan entonces el alcance y la carga útil, y para trabajar cerca del alcance máximo conviene usar servos de engranajes metálicos en el hombro y el codo. La separación de las fuentes, con tierra común, aísla a la ESP32 del ruido de los servos.

6. REFERENCIAS

[1]SU, Haijun. *Robot Kinematics* [en línea]. Columbus: The Ohio State University, 2026. Disponible en: https://cdn.jsdelivr.net/gh/haijunsu-osu/lec_robot_kinematics@main/robot_kinematics.pdf

[2]ASADA, H. y LEONARD, J. *2.12 Introduction to Robotics, cap. 4: Planar Kinematics* [en línea]. Cambridge: MIT OpenCourseWare, 2005. Disponible en: https://ocw.mit.edu/courses/2-12-introduction-to-robotics-fall-2005/resources/chapter4/

[3]*Hojas de datos de los servomotores SG90 y MG90S* [en línea]. [s. f.].

[4]NXP SEMICONDUCTORS. *PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller*: hoja de datos. [s. f.].

[5]ESPRESSIF SYSTEMS. *ESP32 Series Datasheet*. [s. f.].

[6]ESPRESSIF SYSTEMS. *WiFi API*. Arduino-ESP32 documentation [en línea]. Disponible en: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[7]*Hoja de datos del sensor ultrasónico HC-SR04* [en línea]. [s. f.].

[8]GOOGLE. *Android Developers* [en línea]. Disponible en: https://developer.android.com

ANEXO I. RUTINA DE CÓDIGO

I.1 Descripción

La función `ejecutarRutinaApilado()` revisa tres posiciones con el sensor HC-SR04 y apila en una pila fija los cubos que encuentra. Cada punto se convierte en ángulos de servo con la cinemática inversa del Anexo II. La secuencia es:

1. Validar la configuración y la alcanzabilidad de los puntos y tramos (`configuracionLista()` y `validarRutina()`).
2. Abrir la pinza e ir a la pose de escaneo de la posición 1.
3. Medir la distancia en cada posición (mediana de cinco lecturas) y comparar con el umbral.
4. Si hay objeto, ejecutar `transportarObjeto()`: bajar, cerrar la pinza, subir, ir sobre la pila, bajar al nivel, soltar y subir.
5. Pasar a la posición siguiente; al terminar, volver al inicio y devolver `RUTINA_OK`.

Si llega STOP o un punto resulta inalcanzable, la rutina se interrumpe y devuelve `RUTINA_DETENIDA`, `RUTINA_ALCANCE` o `RUTINA_ERROR`.

I.2 Parámetros

Tabla 3. Parámetros de la rutina

Parámetro	Valor	Uso
`CANTIDAD_POSICIONES`	3	Posiciones de recogida
`POSICION_X / POSICION_Y` (mm)	(165,5; 0), (150; 70), (106,4; 126,8)	Centro de cada cubo
`PILA_X / PILA_Y` (mm)	(72,7; −155,9)	Ubicación de la pila
`DIMENSION_CUBO` (mm)	20	Altura de cada nivel
`CUBO_Z_CENTRO` (mm)	10	Centro de un cubo sobre la mesa
`UMBRAL_CM` (cm)	7,86	Distancia máxima para detectar un cubo
`ALTURA_APROXIMACION_MM` (mm)	22	Altura sobre el punto de agarre
`ALTURA_APROXIMACION_PILA_MM` (mm)	55	Altura sobre el depósito de cada nivel
`MARGEN_DEPOSITO_MM` (mm)	5	Holgura al soltar el cubo
`PASO_TRAYECTORIA_MM` (mm)	2	Distancia entre puntos intermedios
`MAX_SALTO_ARTICULAR_GRADOS` (°)	10	Cambio máximo de un servo entre pasos
`PINZA_ABIERTA / PINZA_CERRADA` (°)	26 / 86	Ángulos de la pinza

I.3 Código

Extracto de las funciones principales; se omitieron las funciones auxiliares y los mensajes de error.

```
Punto puntoEnTrayectoria(const Punto& a, const Punto& b, float f) {
    const float la = radioDe(a);
    const float lb = radioDe(b);
    float ta = atan2f(a.y, a.x);
    if (la < 1e-3f) ta = atan2f(b.y, b.x);

    const float l = la + f * (lb - la);            // radio interpolado
    const float t = ta + f * giroBase(a, b);       // giro de la base
    Punto p = {l * cosf(t), l * sinf(t), a.z + f * (b.z - a.z)};
    return p;
}

bool moverEnTrayectoria(const Punto& destino, int pausaMs) {
    if (detenerRutina) return false;

    Punto inicio;
    if (!obtenerPuntoActual(inicio)) return false;      // sin calibración

    int pasos = (int)ceilf(longitudTrayectoria(inicio, destino) /
            PASO_TRAYECTORIA_MM);
    if (pasos < 1) pasos = 1;

    for (int i = 1; i <= pasos; i++) {
        if (paradaSolicitada()) return false;

        float t = (1.0f - cosf(PI * i / pasos)) / 2.0f;   // arranque y frenado
                suaves
        Punto p = puntoEnTrayectoria(inicio, destino, t);

        float servo[CANTIDAD_ARTICULACIONES];
        if (calcularAngulosServo(p, servo) != IK_OK) return false;   // fuera de
                alcance o de límites

        for (int j = 0; j < CANTIDAD_ARTICULACIONES; j++) {
            if (fabsf(servo[j] - poseActual.angulos[j]) >
                    MAX_SALTO_ARTICULAR_GRADOS) return false;
        }
        for (int j = 0; j < CANTIDAD_ARTICULACIONES; j++) {
            poseActual.angulos[j] = servo[j];
            posicionarServo(j, servo[j]);
        }
        delay(pausaMs);
        yield();
    }
    return true;
}

bool transportarObjeto(int posicion, int nivel) {
    nivel = constrain(nivel, 0, CANTIDAD_POSICIONES - 1);

    // 1. Alinear la garra sobre el cubo y bajar
    if (!moverEnTrayectoria(puntoAproximacion(posicion), PAUSA_PASO_LIBRE_MS))
            return false;
    if (!moverEnTrayectoria(puntoRecogida(posicion), PAUSA_PASO_LIBRE_MS)) return
            false;

    // 2. Tomar (cerrar pinza)
    moverServoSuavemente(PINZA, PINZA_CERRADA, PAUSA_PASO_ARTICULAR_MS);
    esperarMs(PAUSA_AGARRE_MS);

    // 3. Subir con el cubo
    if (!moverEnTrayectoria(puntoAproximacion(posicion), PAUSA_PASO_TRANSPORTE_MS))
            return false;

    // 4. Ir sobre la pila
    if (!moverEnTrayectoria(puntoSobrePila(nivel), PAUSA_PASO_TRANSPORTE_MS)) return
            false;
```

```
    // 5. Bajar al nivel de depósito
    if (!moverEnTrayectoria(puntoDeposito(nivel), PAUSA_PASO_TRANSPORTE_MS)) return
            false;

    // 6. Soltar el cubo
    moverServoSuavemente(PINZA, PINZA_ABIERTA, PAUSA_PASO_ARTICULAR_MS);
    esperarMs(PAUSA_AGARRE_MS);

    // 7. Subir libre de la pila
    if (!moverEnTrayectoria(puntoSobrePila(nivel), PAUSA_PASO_LIBRE_MS)) return
            false;

    return !detenerRutina;
}

ResultadoRutina ejecutarRutinaApilado() {
    if (!configuracionLista()) return RUTINA_CONFIG;
    if (!validarRutina()) return RUTINA_ALCANCE;

    estadoActual = "RUTINA";
    cubosApilados = 0;
    posicionActual = 0;
    bool error = false;

    // Pinza abierta y pose de escaneo de la posición 1
    moverServoSuavemente(PINZA, PINZA_ABIERTA, PAUSA_PASO_ARTICULAR_MS);
    if (!moverAPuntoArticular(puntoEscaneo(0))) error = true;

    for (int n = 0; n < CANTIDAD_POSICIONES && !error && !detenerRutina; n++) {
        posicionActual = n + 1;
        esperarMs(PAUSA_ESCANEO_MS);
        float distancia = medirDistanciaCm();
        ultimaDistanciaCm = distancia;

        if (hayObjeto(distancia)) {
            if (!transportarObjeto(n, cubosApilados)) {
                error = true;
                break;
            }
            cubosApilados++;
        }

        // Pasar a la siguiente posición (haya o no objeto)
        if (n < CANTIDAD_POSICIONES - 1) {
            if (!moverEnTrayectoria(puntoEscaneo(n + 1), PAUSA_PASO_LIBRE_MS)) error
                    = true;
        }
    }

    if (detenerRutina) return RUTINA_DETENIDA;
    if (error) return RUTINA_ERROR;

    volverAInicio();
    if (detenerRutina) return RUTINA_DETENIDA;
    return RUTINA_OK;
}
```

ANEXO II. DESARROLLO DE ALGORITMO D-H PARA MANIPULADOR ROBÓTICO

II.1 Parámetros y convención

El brazo tiene tres articulaciones rotacionales (base q_1, hombro q_2 y codo q_3) y un tramo fijo hasta el punto de agarre. La pinza se acciona aparte y no forma parte de la cadena. Se aplica la convención D-H estándar, con la matriz de la Ec. (1) y las medidas de la Tabla 4.

Tabla 4. Medidas del manipulador

Símbolo	Valor	Significado
d_1	128,8 mm	Altura del eje del hombro sobre la mesa
a_2	71,8 mm	Eje del hombro al eje del codo
a_3	51,5 mm	Eje del codo al inicio del tramo de la garra
d_x	87 mm	Tramo de la garra a lo largo del antebrazo
d_h	13 mm	Distancia del punto de agarre bajo la línea del antebrazo
a_4	87,97 mm	Tramo fijo de la garra: $\sqrt{(d_x^2 + d_h^2)}$
φ	8,50°	Inclinación del tramo de la garra

El desfase de la garra en distancia (d_x) y en altura (d_h) se trata como un último eslabón fijo de longitud $a_4 = \sqrt{(d_x^2 + d_h^2)}$, inclinado $-\varphi$, con $\varphi = \text{atan2}(d_h, d_x)$. El ángulo de servo se obtiene de cada ángulo articular con servo = desfase + signo·q, con desfases de 90°, 70° y 77° y signos +1, −1 y +1 para base, hombro y codo.

II.2 Tabla D-H y matriz de transformación

Tabla 5. Parámetros D-H del manipulador

Eslabón	θ_i	d_i (mm)	a_i (mm)	α_i
1 (base)	q_1	128,8	0	90°
2 (hombro)	q_2	0	71,8	0°
3 (codo)	q_3	0	51,5	0°
4 (garra, fija)	$-\varphi = -8{,}50°$	0	87,97	0°

La posición del punto de agarre resulta del producto de las cuatro matrices:

$$T_{04} = A_1\, A_2\, A_3\, A_4 \tag{2}$$

Con $c_1 = \cos q_1$, $s_1 = \sin q_1$ y los ángulos sumados $q_2 + q_3$, la posición (x, y, z) de T_{04} se reduce a la Ec. (3), con R el radio horizontal desde el eje de la base y Z la altura sobre el hombro:

$$\begin{aligned} R &= a_2 \cos q_2 + (a_3 + d_x)\cos(q_2 + q_3) + d_h \sin(q_2 + q_3) \\ Z &= a_2 \sin q_2 + (a_3 + d_x)\sin(q_2 + q_3) - d_h \cos(q_2 + q_3) \\ x &= c_1\, R, \quad y = s_1\, R, \quad z = d_1 + Z \end{aligned} \tag{3}$$

II.3 Algoritmo

Cinemática directa. Dados q_1, q_2 y q_3:

1. Calcular A_1, A_2, A_3 y A_4 con la Ec. (1) y los parámetros de la Tabla 5.
2. Acumular $T_1 = A_1$, $T_2 = T_1\,A_2$, $T_3 = T_2\,A_3$ y $T_4 = T_3\,A_4$.
3. Tomar la cuarta columna de cada matriz: T_1 es el hombro, T_2 el codo, T_3 la muñeca y T_4 el punto de agarre. Estos cuatro puntos son los que el robot informa en el mensaje DH.

Cinemática inversa (codo arriba). Dado el punto de agarre (x, y, z):

1. Base: $q_1 = \mathrm{atan2}(y, x)$. Radio $l = \sqrt{(x^2 + y^2)}$ y altura relativa al hombro $z_r = z - d_1$.
2. Distancia del hombro al punto: $h = \sqrt{(l^2 + z_r^2)}$. Elevación: $\alpha = \mathrm{atan2}(z_r, l)$.
3. Del codo al punto de agarre hay una distancia fija $L_3 = \sqrt{((a_3 + d_x)^2 + d_h^2)} = 139{,}11$ mm, con inclinación $\delta = \mathrm{atan2}(-d_h, a_3 + d_x) = -5{,}36°$. Con $L_2 = a_2$, el teorema del coseno da la Ec. (4).

$$\cos\beta = (L_2^2 + h^2 - L_3^2) \,/\, (2\,L_2\,h) \tag{4}$$

4. Si $|\cos\beta| > 1$, el punto está fuera de alcance. En caso contrario: $q_2 = \alpha + \beta$.
5. Dirección del codo al punto: $\psi = \mathrm{atan2}(z_r - L_2 \sin q_2,\, l - L_2 \cos q_2)$. Codo: $q_3 = \psi - q_2 - \delta$.
6. Convertir cada q a ángulo de servo y comprobar los límites; si alguno se excede, el punto se rechaza.

II.4 Resultados

Para la pose inicial (servos en 80°, 80° y 80°, es decir, $q = (-10°, -10°, 3°)$), la cadena entrega las posiciones de la Tabla 6.

Tabla 6. Posiciones de las articulaciones en la pose inicial
(coordenadas en mm)

Punto	x	y	z
Hombro (T_1)	0,00	0,00	128,80
Codo (T_2)	69,64	−12,28	116,33
Muñeca (T_3)	119,97	−21,15	110,06
Garra (T_4)	203,45	−35,87	86,55

La matriz de la garra en esa pose es:

$$\begin{bmatrix} 0{,}9490 & 0{,}2632 & -0{,}1736 & 203{,}45 \\ -0{,}1673 & -0{,}0464 & -0{,}9848 & -35{,}87 \\ -0{,}2672 & 0{,}9636 & 0{,}0000 & 86{,}55 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La Ec. (3) coincide con el producto de matrices: con q = (30°, 40°, −25°), ambos métodos entregan (166,40; 96,07; 198,24) mm.